IPA 1/69

Expressions, Functions & Iteration

WEEK 13

for_each

for

dynamic

try

Week 13 Agenda

- 4.1 — Expanding the module: names, tags, multiple instances, IAM role, SSM
- 4.2 — Operators & conditionals: math, comparison, boolean, ternary
- 4.3 — Functions: calling functions, standard library, pure vs. impure
- 4.4 — Strings & templates: file(), templatefile(), template language
- 4.5 — Regular expressions: regex, regexall, replace
- 4.6 — Type conversion: implicit, toType(), sensitive/nonsensitive
- 4.7 — try and can: safe expression evaluation
- 4.8 — count and for_each: differences, limitations, workarounds
- 4.9 — for expressions: list/object outputs, filtering, splat
- 4.10 — Dynamic blocks: generating repeated nested blocks
- **LAB: Multi-instance config with for_each, dynamic SG, templatefile**

Operators & Conditionals

Arithmetic, comparison, logic, and ternary

Arithmetic

`+ - * /`	Add, subtract, multiply, divide
`%`	Modulo (remainder)
`-x`	Negate (unary minus)

Note: integers only — no float division

Comparison

`== !=`	Equal / not equal
`< > <= >=`	Numeric / string comparison
`Returns bool`	Always true or false

Boolean / Logic

`&&`	AND — both must be true
`||`	OR — either must be true
`!`	NOT — inverts bool
`Short-circuits`	Like most languages

Conditional (Ternary)

```
condition ? true_val : false_val

var.env == "prod"          ? "t3.large"
                           : "t3.micro"
```

Works with any type, both branches same type

Functions

Terraform's built-in standard library

String Functions

```
upper("hello")          → "HELLO"
lower("WORLD")          → "world"
trimspace("  hi  ")     → "hi"
replace("a-b","a","x")  → "x-b"
split(",","a,b,c")      → ["a","b","c"]
join("-",["a","b"])     → "a-b"
format("%.2f", 3.1)     → "3.10"
```

Collection Functions

```
length(["a","b"])       → 2
concat(["a"],["b"])     → ["a","b"]
flatten([["a"],["b"]])  → ["a","b"]
keys({a=1,b=2})         → ["a","b"]
values({a=1,b=2})       → [1,2]
merge({a=1},{b=2})      → {a=1,b=2}
toset(["a","a","b"])    → {"a","b"}
```

Numeric / Type Functions

```
max(1,2,3)              → 3
min(1,2,3)              → 1
floor(2.9)              → 2
ceil(2.1)               → 3
tostring(42)            → "42"
tonumber("3.14")        → 3.14
tobool("true")          → true
```

Filesystem / Encode Functions

```
file("./script.sh")     reads file to string
templatefile(path,vars) render template
jsonencode({a=1})       → "{"a":1}"
jsondecode(str)         → object
base64encode("hello")   → "aGVsbG8="
md5("data")             → hash string
timestamp()             → "2024-01-15T..."
```

Strings & Templates

file(), templatefile(), and the template language

```
# Inline interpolation
"Hello, ${var.name}!"

# file() — read raw file content
user_data = file("./scripts/bootstrap.sh")

# templatefile() — render with variables
user_data = templatefile("./tpl/init.sh.tpl", {
  db_host = aws_db_instance.main.address
  env     = var.environment
})
```

```
# init.sh.tpl — template file
#!/bin/bash
DB_HOST="${db_host}"    # interpolation

%{ for item in items ~}
  echo "Processing ${item}"
%{ endfor ~}

%{ if env == "prod" ~}
  enable_monitoring=true
%{ endif ~}
```

`${...}`

Expression interpolation — inserts value into string

`%{ for x in list ~}`

Template for loop. Trailing ~ strips whitespace.

`%{ if cond ~}`

Template conditional. Nesting supported.

`file() vs templatefile()`

file() = raw string. templatefile() = rendered with vars dict.

When NOT to use

Complex logic → use locals/variables instead of templates. Templates are for user_data, config files, scripts.

`template_file (legacy)`

Old provider-based approach. Use templatefile() built-in instead.

count and for_each

Create multiple resource instances

count — integer-based

```
resource "aws_instance" "web" {
  count         = 3
  ami           = var.ami_id
  instance_type = var.instance_type
  tags = {
    Name = "web-${count.index}"
  }
}

# Address: aws_instance.web[0]
# Address: aws_instance.web[*]  (all)
```

- Use when: identical copies, order matters
- count.index = 0,1,2,...
- Removing middle element renumbers ALL → recreate
- Avoid count with dynamic values (known after apply)

for_each — map or set-based (preferred)

```
variable "instances" {
  type = map(string)
  default = {
    web = "t3.micro"
    api = "t3.small"
  }
}
resource "aws_instance" "app" {
  for_each      = var.instances
  ami           = var.ami_id
  instance_type = each.value
  tags = { Name = each.key }
}
```

- each.key = map key or set element
- each.value = map value (or same as key for set)
- Address: aws_instance.app["web"]
- Removing one key → only that instance deleted
- for_each needs known-at-plan values (no computed)

for Expressions

Transform and filter collections inline

```
# List → list  (square brackets)
[for s in var.names : upper(s)]
# → ["ALICE","BOB","CAROL"]

# List → list with filter
[for s in var.names : s if length(s) > 3]
# → ["ALICE","CAROL"]  (Bob filtered out)

# Map → map  (curly braces + => )
{for k,v in var.tags : k => upper(v)}
# → { Env = "PROD", App = "WEB" }

# Splat operator  (shorthand for_each .attr)
aws_instance.web[*].id
# equivalent to:
[for i in aws_instance.web : i.id]

# Grouping mode  (collect duplicates)
{for s in var.items : s.type => s... }
# Groups items by type into lists
```

`[ ... ]`

Produces a list. Key-value syntax not allowed.

`{ key => val }`

Produces a map/object. Both key and value required.

`if clause`

Filter: only items where condition is true are included.

`Splat [*]`

Legacy syntax. Works only on lists, not maps. Prefer for expression.

`Grouping ...`

Ellipsis after value → collect all matches into a list per key.

`Usage in outputs`

Common pattern: output a map of name → id for downstream use.

`Nesting`

for expressions can nest, but keep readable — extract to locals.

Dynamic Blocks

Generate repeated nested blocks from a collection

```hcl
variable "ingress_ports" {
  type = list(object({
    port     = number
    protocol = string
  }))
  default = [
    { port = 80,  protocol = "tcp" },
    { port = 443, protocol = "tcp" },
  ]
}
resource "aws_security_group" "web" {
  dynamic "ingress" {
    for_each = var.ingress_ports
    content {
      from_port   = ingress.value.port
      to_port     = ingress.value.port
      protocol    = ingress.value.protocol
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
}
```

try & can (Ch. 4.7): try(expr, fallback) returns fallback if expr errors. can(expr) returns bool. Use for optional/nullable expressions.

dynamic "<block_name>" { }

Creates one nested block per element of for_each

for_each

Any collection: list, set, or map of objects

content { }

The body of each generated nested block

iterator

Default label is the block name. Override with iterator = "port"

.value / .key

Access each element's value (or key for maps)

When to use

Security group rules, ingress/egress, tags blocks, IAM statements

When NOT to use

If you only need 1-2 blocks, hardcode them — dynamic adds complexity

HANDS-ON LAB

Week 3 Lab — Multi-Instance Config with Iteration & Templates

Objective: Deploy multiple resources dynamically using variables, expressions, and templates

1

for_each instances

Map variable: name → type
Create multiple EC2 instances
Tag each with each.key

2

Dynamic security group

List of port objects variable
dynamic "ingress" block
Verify rules created correctly

3

templatefile()

Write init.sh.tpl template
Pass db_host, env variables
Verify rendered user_data

4

for expression outputs

Output map: name → instance_id
[for k,v in instances : ...]
Verify with terraform output

5

IAM role + SSM

aws_iam_role + instance profile
Attach AmazonSSMManagedInstanceCore
Connect via Session Manager

6

Ternary conditional

instance_type by env:
var.env == "prod"
  ? "t3.large" : "t3.micro"

Deliverable: Multi-instance config with templated user-data and dynamic security group

Week 3 Lab — Iteration & Templates (AWS Academy)

Objective: for_each, dynamic blocks, templatefile — use LabInstanceProfile instead of creating IAM role

1

for_each map

{ web="t3.micro", api="t3.small" }
EC2 per key, tagged with
each.key (max t3.large)

2

Dynamic SG

List of port objects
dynamic "ingress" block
Verify rules in AWS Console

3

templatefile()

Write init.sh.tpl
Pass db_host, env vars
Verify rendered user_data

4

IAM — LabProfile

data "aws_iam_instance_profile"
{ name = "LabInstanceProfile" }
iam_instance_profile = .name

5

Session Manager

LabRole has SSM policy
No additional IAM needed
Connect via SSM in Console

6

for output map

[for k,v in instances
: k => v.id]
Verify terraform output

Week 13 — Key Takeaways

WEEK 13

What you should be able to do after today

Operators & Ternary

condition ? val_if_true : val_if_false — great for environment-based sizing.

Functions

Terraform's standard library covers strings, collections, numbers, encoding, filesystem. No custom functions.

templatefile()

Render scripts with variables. Use %{ for } and %{ if } in .tpl files. Better than heredoc for complex scripts.

count vs for_each

count for identical clones; for_each (map/set) when each has a stable key. for_each is almost always better.

for expressions

[for x in list : transform(x) if filter(x)] — list output. {for k,v : k => v} — map output.

dynamic blocks

dynamic { for_each + content { } } generates repeated nested blocks (security group rules, IAM statements).

Next week: Plan/apply lifecycle, dependency graphs, state basics → Ch. 5 + 6.1–6.3